Guide d'Exécution Complet

Ordre de Lancement des Fichiers DuraXELL

Chaque fichier expliqué : ce qu'il fait, quand le lancer, quelles données il consomme, ce qu'il produit.

https://github.com/longeacc/DuraXell

Mars 2026

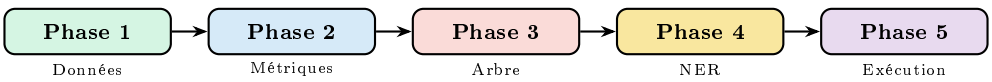


Table des matières

1 Vue d'Ensemble : les 6 Phases du Pipeline **2**

2 Phase 0 — Préparation des Données **3**
2.1 `ESMO2025/Rules/src/Breast/biomarker_brat_annotator.py` . . . 3
2.2 `ESMO2025/Rules/src/Breast/lunch.py` . . . 3

3 Phase 1 — Calcul des Métriques Primaires **4**
3.1 Étape 1.1 — `ESMO2025/E_templeability.py` . . . 4
3.2 Étape 1.2 — `ESMO2025/E_homogeneity.py` . . . 4
3.3 Étape 1.3 — `ESMO2025/E_risk_context.py` . . . 4
3.4 Étape 1.4 — `ESMO2025/E_frequency.py` . . . 4
3.5 Étape 1.5 — `ESMO2025/E_annotation_yield.py` . . . 5

4 Phase 2 — Métriques Dérivées et Arbre **6**
4.1 Étape 2.1 — `ESMO2025/E_fesability_NER.py` . . . 6
4.2 Étape 2.2 — `ESMO2025/E_creation_arbre_decision.py` . . . 6
4.3 Étape 2.3 (optionnel) — `ESMO2025/visualize_decision_tree.py` . . . 6

5 Phase 3 — Entraînement NER (optionnel) **7**
5.1 Étape 3.1 — `NER/src/1convert_brat_to_conll.py` . . . 7
5.2 Étape 3.2 — `NER/src/2sweep_ner.py` . . . 7
5.3 Étape 3.2bis — `NER/src/2bis_train_hf_ner.py` . . . 7
5.4 Étape 3.3 — `NER/src/3infer.py` . . . 7
5.5 Étape 3.4 — `NER/src/4predict_to_brat.py` . . . 7
5.6 Étape 3.5 — `NER/src/5evaluate_ner.py` . . . 8
5.7 `NER/src/eval_best_model.py` (raccourci) . . . 8
5.8 `NER/src/ner_cascade_connector.py` (connecteur) . . . 8

6 Phase 4 — Exécution de la Cascade **9**
6.1 Le point d'entrée principal : `main.py` . . . 9
6.2 `ESMO2025/cascade_orchestrator.py` . . . 9
6.3 Fichiers support de la cascade . . . 9

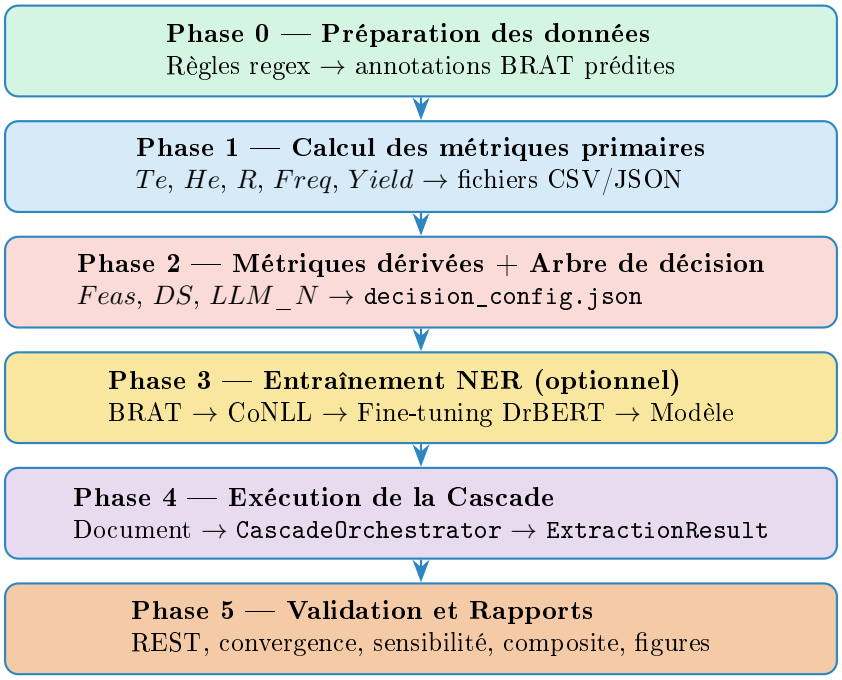
7 Phase 5 — Validation et Rapports **10**
7.1 `run_full_pipeline_report.py` . . . 10
7.2 `ESMO2025/E_composite_scorer.py` . . . 10
7.3 `ESMO2025/sensitivity_analysis.py` . . . 10
7.4 Module REST (validation bottom-up) . . . 10
7.5 Générateurs de rapports HTML . . . 11

8 Fichiers Annexes **12**
8.1 Tests unitaires (`ESMO2025/tests/`) . . . 12
8.2 Fichiers de visualisation et documentation . . . 12

9 Résumé : Séquence Complète de Lancement **13**

1 Vue d'Ensemble : les 6 Phases du Pipeline

Le pipeline DuraXELL s'exécute en **6 phases séquentielles**. Chaque phase dépend des sorties de la précédente. On peut relancer une phase sans relancer les précédentes si les fichiers de sortie existent déjà.



2 Phase 0 — Préparation des Données

Phase 0 : Génération des annotations prédites par les règles

But : Appliquer les règles regex sur le corpus pour générer des annotations au format BRAT. Ces prédictions serviront de référence pour calculer le Yield et pour comparer aux annotations manuelles (Gold Standard).

2.1 `ESMO2025/Rules/src/Breast/biomarker_brat_annotator.py`

- **Ce qu'il fait :** Contient les règles regex pour chaque biomarqueur du cancer du sein (ER, PR, HER2, Ki67, etc.). Lit des fichiers `.txt` cliniques et génère des fichiers `.ann` au format BRAT.
- **Quand le lancer :** En tout premier, avant toute métrique.
- **Entrée :** `Dossiers de fichiers .txt cliniques`
- **Sortie :** `Fichiers .ann (annotations BRAT prédites)`

2.2 `ESMO2025/Rules/src/Breast/lunch.py`

- **Ce qu'il fait :** Script « lanceur » qui appelle `biomarker_brat_annotator.py` sur 3 corpus : (1) training set, (2) evaluation set RCP, (3) evaluation set CHIR. Produit les dossiers de prédictions.
- **Quand le lancer : Étape 0.1** — après avoir placé les corpus dans `Rules/src/Breast/`.
- **Commande :** `cd ESMO2025/Rules/src/Breast/ && python lunch.py`
- **Entrée :** `Breast/RCP/training_set_breast_cancer/*.txt`
- **Sortie :** `Breast/RCP/evaluation_set_breast_cancer_pred_rules/*.ann`

3 Phase 1 — Calcul des Métriques Primaires

Phase 1 : Calcul de Te, He, R, $Freq$, $Yield$

But : Calculer les 5 métriques primaires à partir du corpus annoté. Chaque script lit les fichiers BRAT et produit un fichier de résultats dans `ESMO2025/Rules/Results/`.

Les 5 scripts peuvent être lancés dans n'importe quel ordre car ils sont indépendants. L'ordre ci-dessous est logique mais pas obligatoire.

3.1 Étape 1.1 — `ESMO2025/E_templeability.py`

- **Ce qu'il fait :** Calcule le score de Templateabilité (Te) pour chaque type d'entité. Normalise les mentions en patterns abstraits, calcule l'entropie de Shannon normalisée, ajoute un bonus sémantique.
- **Commande :** `python ESMO2025/E_templeability.py`
- **Entrée :** `Corpus BRAT : NER/data/Breast/{train,val,test}/*.ann`
- **Sortie :** `ESMO2025/Rules/Results/templeability_analysis.json`

3.2 Étape 1.2 — `ESMO2025/E_homogeneity.py`

- **Ce qu'il fait :** Calcule l'Homogénéité (He) pour chaque entité. Tokenise les mentions, calcule le ratio de redondance brute $(T-U)/T$, applique la transformation sigmoid ($k = 10$, $x_0 = 0.5$).
- **Commande :** `python ESMO2025/E_homogeneity.py`
- **Entrée :** `Corpus BRAT : NER/data/Breast/{train,val,test}/*.ann`
- **Sortie :** `ESMO2025/Rules/Results/homogeneity_analysis.csv`

3.3 Étape 1.3 — `ESMO2025/E_risk_context.py`

- **Ce qu'il fait :** Calcule le Risque Contextuel (R) pour chaque entité. Lit les fichiers `.ann` ET `.txt` pour extraire une fenêtre de ± 50 caractères autour de chaque mention, détecte négation (10 patterns), incertitude (14 patterns), contradiction (positif+négatif dans le même document).
- **Commande :** `python ESMO2025/E_risk_context.py`
- **Entrée :** `Corpus BRAT : NER/data/Breast/{train,val,test}/*.ann + .txt`
- **Sortie :** `Results/risk_context_analysis.csv`

3.4 Étape 1.4 — `ESMO2025/E_frequency.py`

- **Ce qu'il fait :** Calcule la Fréquence ($Freq$) relative de chaque entité. Compte les tokens du corpus (dénominateur) et les annotations par type (numérateur). Produit aussi un histogramme ASCII en échelle log.
- **Commande :** `python ESMO2025/E_frequency.py`
- **Entrée :** `Corpus BRAT : NER/data/Breast/{train,val,test}/*.ann + .txt`
- **Sortie :** `ESMO2025/Rules/Results/frequency_analysis.csv`

3.5 Étape 1.5 — `ESMO2025/E_annotation_yield.py`

- **Ce qu'il fait :** Calcule le Rendement d'Annotation (*Yield*) en comparant les prédictions des règles (Phase 0) au Gold Standard humain. Utilise un matching par IoU$\geq$0.5 (greedy one-to-one) puis calcule Précision, Rappel, F_1 par entité.
- **Commande :** `python ESMO2025/E_annotation_yield.py`
- **Entrée :** `Gold Standard : evaluation_set_breast_cancer_GS/*.ann`
- **Entrée :** `Prédictions règles : evaluation_set_breast_cancer_pred_rules/*.ann`
- **Sortie :** `Scores affichés (pas de CSV automatique -- intégré via l'arbre)`

Dépendance : Nécessite les sorties de la Phase 0 (`lunch.py`).

4 Phase 2 — Métriques Dérivées et Arbre

Phase 2 : $Feas$, DS, LLM_N, Construction de l'arbre

Dépendance : Nécessite TOUS les CSV/JSON de la Phase 1.

4.1 Étape 2.1 — `ESMO2025/E_fesability_NER.py`

- **Ce qu'il fait :** Agrège les métriques primaires (Te, He, R, $Freq$, $Yield$) pour calculer 3 métriques dérivées : Faisabilité NER ($Feas$), Décalage de Domaine (DS, avec MMD), Nécessité LLM (LLM_N).
- **Commande :** `python ESMO2025/E_fesability_NER.py`
- **Entrée :** `Tous les fichiers de Phase 1 :`
  `Rules/Results/templeability_analysis.json`
  `Rules/Results/homogeneity_analysis.csv`
  `Rules/Results/frequency_analysis.csv`
  `Results/risk_context_analysis.csv`
- **Sortie :** `ESMO2025/Rules/Results/ner_feasibility_analysis.csv`

4.2 Étape 2.2 — `ESMO2025/E_creation_arbre_decision.py`

- **Ce qu'il fait :** Charge toutes les métriques (primaires + dérivées), exécute l'arbre de décision à 6 nœuds pour chaque entité, effectue la validation k-fold des seuils, génère la configuration machine-readable et un rapport textuel.
- **Commande :** `python ESMO2025/E_creation_arbre_decision.py`
- **Entrée :** `Tous les CSV/JSON des Phases 1 et 2.1`
- **Sortie :** `decision_config.json (fichier central du projet)`
- **Sortie :** `output_decision.txt (rapport lisible)`

Ce fichier est le **pivot** du pipeline : il produit le `decision_config.json` consommé par le `CascadeOrchestrator` en Phase 4.

4.3 Étape 2.3 (optionnel) — `ESMO2025/visualize_decision_tree.py`

- **Ce qu'il fait :** Génère une image PNG de l'arbre de décision via NetworkX/Matplotlib.
- **Commande :** `python ESMO2025/visualize_decision_tree.py`
- **Entrée :** `decision_config.json`
- **Sortie :** `Results/figures/decision_tree_visualization.png`

5 Phase 3 — Entraînement NER (optionnel)

Phase 3 : Pipeline NER Transformer (DrBERT fine-tuning)

Cette phase est optionnelle. Elle n'est nécessaire que si l'arbre de décision recommande un Transformer pour certaines entités. Le pipeline est numéroté 1–5 pour l'ordre d'exécution.

5.1 Étape 3.1 — `NER/src/1convert_brat_to_conll.py`

- **Ce qu'il fait :** Convertit les annotations BRAT (`.ann + .txt`) en format CoNLL (BIO-tagging). Chaque token reçoit une étiquette B-Entity, I-Entity, ou O.
- **Commande :** `python NER/src/1convert_brat_to_conll.py`
- **Entrée :** `Corpus BRAT : NER/data/Breast/{train,val,test}/`
- **Sortie :** `Fichiers .conll (un par split)`

5.2 Étape 3.2 — `NER/src/2sweep_ner.py`

- **Ce qu'il fait :** Recherche d'hyperparamètres par grid search. Teste des combinaisons de learning rate, batch size, et nombre d'epochs pour trouver la meilleure configuration.
- **Commande :** `python NER/src/2sweep_ner.py`
- **Entrée :** `Fichiers CoNLL`
- **Sortie :** `NER/models/sweeps/ (checkpoints pour chaque combinaison)`

5.3 Étape 3.2bis — `NER/src/2bis_train_hf_ner.py`

- **Ce qu'il fait :** Fine-tuning de DrBERT (ou autre modèle HuggingFace) avec les hyperparamètres optimaux identifiés par le sweep. Entraînement complet sur train, évaluation sur val.
- **Commande :** `python NER/src/2bis_train_hf_ner.py`
- **Entrée :** `Fichiers CoNLL + config d'hyperparamètres`
- **Sortie :** `Modèle entraîné dans NER/models/`

5.4 Étape 3.3 — `NER/src/3infer.py`

- **Ce qu'il fait :** Applique le modèle entraîné sur de nouveaux textes (inférence batch).
- **Commande :** `python NER/src/3infer.py`
- **Entrée :** `Modèle entraîné + textes de test`
- **Sortie :** `Prédictions au format CoNLL`

5.5 Étape 3.4 — `NER/src/4predict_to_brat.py`

- **Ce qu'il fait :** Convertit les prédictions CoNLL du modèle en format BRAT pour évaluation visuelle et comparaison.
- **Entrée :** `Prédictions CoNLL`
- **Sortie :** `Fichiers .ann prédits par le NER`

5.6 Étape 3.5 — `NER/src/5evaluate_ner.py`

— **Ce qu'il fait :** Calcule le F_1-score du modèle NER : token-level et entity-level, par type d'entité.

— **Entrée :** `Prédictions + Gold Standard (CoNLL)`

— **Sortie :** `Rapport d'évaluation (précision, rappel,` F_1 `par entité)`

5.7 `NER/src/eval_best_model.py` (raccourci)

Script qui charge directement le meilleur modèle, évalue sur le test set, et affiche le rapport complet (seqeval).

5.8 `NER/src/ner_cascade_connector.py` (connecteur)

— **Ce qu'il fait :** Adaptateur entre le modèle NER et le `CascadeOrchestrator`. Charge le modèle une seule fois, transforme les prédictions HuggingFace en objets `ExtractionResult`.

— **Ne se lance pas seul :** il est importé par `cascade_orchestrator.py`.

6 Phase 4 — Exécution de la Cascade

Phase 4 : Extraction via le CascadeOrchestrator

Dépendance : `decision_config.json` (Phase 2) et optionnellement le modèle NER (Phase 3).

6.1 Le point d'entrée principal : `main.py`

Le fichier `main.py` est le **CLI central** de DuraXELL. Il expose 8 commandes :

TABLE 1 – Commandes du CLI `main.py`

Commande	Usage	Ce qu'elle fait
`extract`	`python main.py extract -doc "..." -entity ER`	Extrait une entité d'un texte
`extract-all`	`python main.py extract-all -doc "..."`	Extrait les 7 entités d'un texte
`batch`	`python main.py batch -input_dir corpus/`	Extraction batch (dossier de `.txt`)
`metrics`	`python main.py metrics`	Lance Te et He
`tree`	`python main.py tree`	Génère l'arbre + visualisation
`rest`	`python main.py rest`	Lance la démo REST
`evaluate`	`python main.py evaluate`	Lance le pipeline complet
`info`	`python main.py info`	Affiche les infos de configuration

6.2 `ESMO2025/cascade_orchestrator.py`

- **Ce qu'il fait :** Charge `decision_config.json`, détermine la méthode recommandée pour chaque entité, et exécute la cascade à 4 niveaux : Rules → CRF → Transformer → LLM. Si un niveau retourne une confiance ≥ 0.7, il s'arrête. Sinon, il escalade au niveau suivant (plafonné par l'arbre).
- **Ne se lance pas seul :** il est importé par `main.py` et `run_full_pipeline_report.py`.
- **Entrée :** `decision_config.json + texte + type d'entité`
- **Sortie :** `Objet ExtractionResult (valeur, méthode, confiance, énergie, temps)`

6.3 Fichiers support de la cascade

Fichier	Rôle
`ESMO2025/structs.py`	Définit `ExtractionResult` (9 champs). Importé partout.
`ESMO2025/energy_tracker.py`	Mesure le kWh via eco2ai. Utilisé par l'orchestrateur.
`Rules/src/Breast/rules_cascade_connector.py`	Adaptateur règles → `ExtractionResult`.
`NER/src/ner_cascade_connector.py`	Adaptateur Transformer → `ExtractionResult`.

7 Phase 5 — Validation et Rapports

Phase 5 : Validation croisée, sensibilité, rapports

7.1 `run_full_pipeline_report.py`

— **Ce qu'il fait :** Script maître qui orchestre la génération complète des rapports. Exécute 6 blocs : (1) Performance (F_1 par méthode), (2) Explicabilité, (3) Énergie, (4) Front de Pareto et radar chart, (5) Analyse de sensibilité, (6) Pont REST.

— **Commande :** `python run_full_pipeline_report.py` (ou `python main.py evaluate`)

— **Entrée :** `decision_config.json + tous les modules`

— **Sortie :** `CSV dans Results/, figures dans Results/figures/`

7.2 `ESMO2025/E_composite_scorer.py`

— **Ce qu'il fait :** Calcule le score composite $C = 0.4F_1 + 0.3Expl + 0.3(1 - E_{\text{norm}})$. Effectue l'analyse de dominance de Pareto. Génère le radar chart comparatif.

— **Ne se lance pas seul :** importé par `run_full_pipeline_report.py`.

7.3 `ESMO2025/sensitivity_analysis.py`

— **Ce qu'il fait :** Perturbe les seuils de l'arbre de ±10–20% et mesure l'impact sur les décisions de routage. Identifie les seuils les plus sensibles.

— **Commande :** `python ESMO2025/sensitivity_analysis.py`

— **Entrée :** `decision_config.json`

— **Sortie :** `Rapport de sensibilité (console)`

7.4 Module REST (validation bottom-up)

Table 2 – Fichiers du module REST et leur ordre

Ordre	Fichier	Ce qu'il fait
5.1	`rest_annotator.py`	Annotation semi-automatique assistée par regex sur un corpus pilote
5.2	`rest_evaluator.py`	Évalue les métriques REST par entité (bottom-up)
5.3	`yield_calculator.py`	Calcule le Yield spécifique REST avec bootstrap
5.4	`rest_decision_bridge.py`	Compare décisions Arbre (top-down) *vs* REST (bottom-up)
5.5	`convergence_analyzer.py`	Génère le rapport de convergence + graphiques
5.6	`rest_pipeline.py`	Orchestre 5.1–5.5 en un seul appel
5.7	`demo_rest.py`	Démo intégrée avec documents simulés

Commande : `python main.py rest` ou `python ESMO2025/REST_interface/demo_rest.py`

7.5 Générateurs de rapports HTML

Fichier	Ce qu'il fait
`generate_templeability_report.py`	Génère un HTML interactif à partir de `templeability_analysis.json`
`generate_homogeneity_report.py`	Génère un HTML depuis `homogeneity_analysis.csv`
`generate_risk_context_report.py`	Génère un HTML depuis `risk_context_full.json + .csv`

8 Fichiers Annexes

8.1 Tests unitaires (`ESMO2025/tests/`)

Fichier	Ce qu'il teste
`conftest.py`	Fixtures pytest partagées (corpus mock, configurations)
`test_templeability.py`	Score Te sur des cas synthétiques
`test_homogeneity.py`	Score He sur des cas synthétiques
`test_risk_context.py`	Score R : négation, incertitude, contradiction
`test_frequency.py`	Comptage et fréquences
`test_annotation_yield.py`	Matching IoU, F_1, cas limites
`test_composite_scorer.py`	Score C, Pareto
`test_decision_tree.py`	Routage de l'arbre sur des métriques simulées
`test_cascade.py`	Orchestrateur (escalade, plafonnement)
`test_energy_tracker.py`	Mesure énergétique
`test_pipeline_integration.py`	Intégration bout-en-bout
`test_simple.py`	Tests de base (imports, instanciation)

Commande : `cd ESMO2025 && pytest tests/ -v`

8.2 Fichiers de visualisation et documentation

Fichier	Ce qu'il fait
`DuraXELL_DependencyTree.jsx`	Composant React interactif de l'arbre de dépendances
`dependency_tree_viewer.html`	Version HTML standalone de la visualisation
`create_notebook.py`	Génère programmatiquement le notebook Jupyter
`Reports/DuraXELL_Pipeline.ipynb`	Notebook interactif pour exécuter le pipeline pas à pas
`commandes.ipynb`	Notebook de référence des commandes CLI
`DEPENDENCY_REPORT.md`	Rapport des dépendances inter-fichiers
`RAPPORT_COMPREHENSION.md`	Notes de compréhension du projet
`graph_orchestrator.py`	Vide (0 octet) — planifié, non implémenté

9 Résumé : Séquence Complète de Lancement

Voici la séquence **minimale** pour faire tourner DuraXELL de zéro sur un nouveau corpus :

Table 3 – Séquence complète de lancement (17 étapes)

N°	Phase	Action	Commande
1	0	Placer le corpus BRAT dans `NER/data/Breast/`	—
2	0	Générer les prédictions des règles	`python lunch.py`
3	1	Calculer Te	`python E_templeability.py`
4	1	Calculer He	`python E_homogeneity.py`
5	1	Calculer R	`python E_risk_context.py`
6	1	Calculer $Freq$	`python E_frequency.py`
7	1	Calculer $Yield$	`python E_annotation_yield.py`
8	2	Calculer $Feas$, DS, LLM_N	`python E_fesability_NER.py`
9	2	Construire l'arbre de décision	`python E_creation_arbre.py`
10	2	Visualiser l'arbre (optionnel)	`python visualize_decision_tree.py`
11	3	Convertir BRAT → CoNLL	`python 1convert_brat_to_conll.py`
12	3	Sweep hyperparamètres (optionnel)	`python 2sweep_ner.py`
13	3	Entraîner le NER	`python 2bis_train_hf_ner.py`
14	3	Évaluer le NER	`python 5evaluate_ner.py`
15	4	Extraire (entité unique)	`python main.py extract ...`
16	4	Extraction batch	`python main.py batch ...`
17	5	Générer tous les rapports	`python main.py evaluate`

Raccourci : Les étapes 3–9 peuvent être partiellement remplacées par `python main.py metrics` (lance Te et He) puis `python main.py tree` (lance l'arbre + visualisation).